

SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM089

Name: Prasad Dinkar Gurav

Division: B

1. System Title & Short Description

Maze Solving System (BFS / DFS) — a search-based agent that finds a path through a grid maze from a start cell to an end cell. The system takes a maze grid (open cells and walls) as input and explores it using either Breadth First Search (BFS) or Depth First Search (DFS). It solves the general path-finding problem: navigating from a known start point to a known goal point while avoiding obstacles, which is a core building block for robotics, games, and route-planning agents. This system continues directly from the BFS/DFS code and profiling work done in SLE-1 and SLE-2.

2. Context Diagram (Level 1)

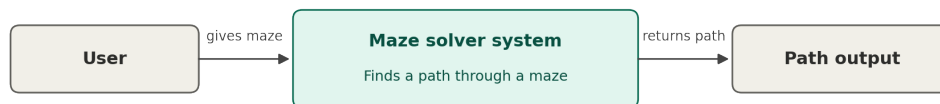


Figure 1: Context diagram – system boundary and external actors

The **User** supplies the maze (the grid layout, start cell, and end cell) to the **Maze solver system**. The system processes this input internally and returns the **Path output** — either the sequence of cells forming the shortest/found path, or a message that no path exists. No external systems are involved; the whole system runs as a self-contained script with no network or API dependency.

3. Container Diagram (Level 2)

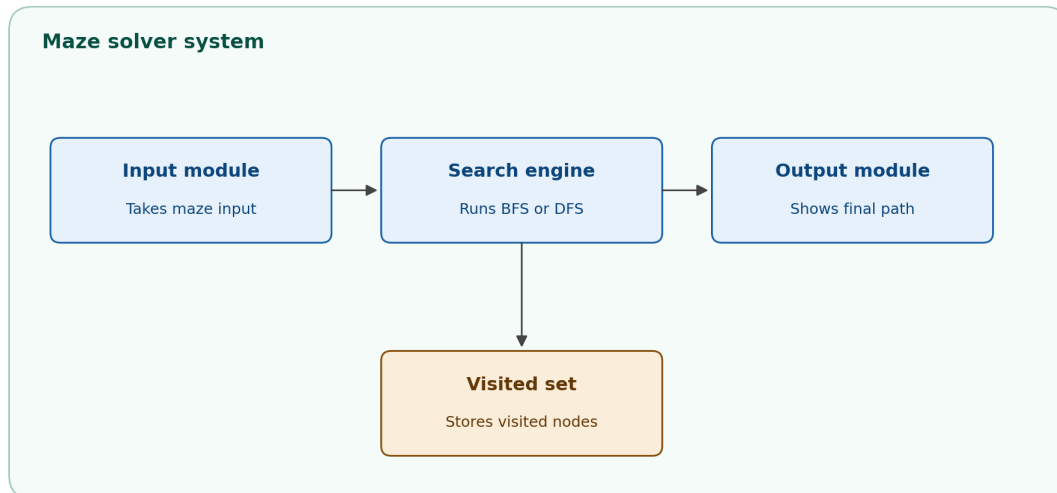


Figure 2: Container diagram – main building blocks inside the system

Explanation of each container:

- **Input module:** Builds/loads the maze grid (start cell, end cell, walls) that the search will run on.
- **Search engine:** Runs the chosen algorithm, BFS or DFS, to explore the maze.
- **Visited set:** Stores which cells have already been explored, so the search never revisits a cell or loops forever.
- **Output module:** Reports the final path (or "no path found"), along with timing/profiling numbers from SLE-2.

4. Component Diagram (Level 3)

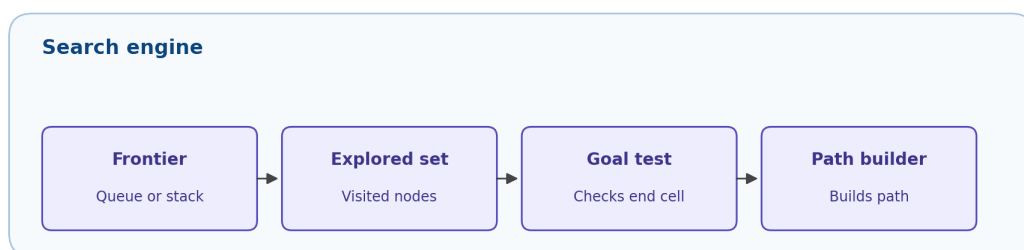


Figure 3: Component diagram – inside the Search engine container

The **Search engine** is the most important container, so it is broken down further. The **Frontier** holds cells waiting to be explored — a Queue for BFS (FIFO) or a Stack for DFS (LIFO). The **Explored set** is the same visited-cells store shown at container level, seen here from inside the engine. The **Goal test** checks whether the current cell is the end cell. The **Path builder** walks the stored parent links backward from the end cell to the start cell once the goal is found, producing the final path.

5. Code Level Overview (Level 4)

Main classes / functions:

- `generate_maze(size)` : builds the maze grid used by both algorithms.
- `bfs(maze, start, end)` : runs Breadth First Search using a queue (frontier).
- `dfs(maze, start, end)` : runs Depth First Search using a stack (frontier).
- `parent dictionary` : records how each cell was reached; shared logic used by both algorithms.
- `Path-reconstruction loop` : walks the parent links back from `end` to `start` to build the final path.

6. Design Decisions

A single `parent dictionary` is reused for path reconstruction in both algorithms instead of separate `Node` objects, keeping the code simple and consistent. The `frontier` is the only structural difference between BFS and DFS (queue vs. stack), which is why the two algorithms can share almost identical surrounding logic (visited-set checks, goal test, path building).

7. AI Contribution Note

- AI tools used (if any): Claude (Anthropic) — used for code generation, profiling, diagram creation, and document formatting.
- What AI helped with: Writing/refining the BFS and DFS maze-solving code and its timing/profiling instrumentation; drawing the Context, Container, and Component (C4) diagrams; and formatting this document.
- What I did myself: Chose the maze-solving system as the project, defined the Input/Search Engine/Visited Set/Output breakdown, decided which container to expand at the Component level, reviewed all AI-generated code and diagrams for correctness, and wrote the design-decision justification.

8. Conclusion

Working through the full C4 model showed how the same system can be described at very different levels of detail without losing consistency — the Context diagram hides the search algorithm completely, while the Code level names the exact functions that implement it. It also made clear that architecture and implementation are separate concerns: BFS and DFS look identical at the Container and Component levels (same Input, Search Engine, Visited Set, Output; same Frontier, Explored Set, Goal Test, Path Builder), and only diverge in one implementation detail — a queue versus a stack. This reinforced why the SLE-2 profiling results made sense: the algorithms differ in performance, not in structure.